

Comparative Study of Label Encoding vs One-Hot Encoding

Objective

To understand and compare two common techniques for converting categorical data into numerical form: Label Encoding (using `sklearn.preprocessing.LabelEncoder`) and One-Hot Encoding (using `pandas.get_dummies()`). The study is performed on a small dummy hospital dataset containing Gender, AppointmentType, and City.

1. Inspect the Dataset

We first create the dataset and examine its structure and content.

```
import pandas as pd

import numpy as np

# Create the dummy dataset

data = {
    "Patient_ID": ["P1", "P2", "P3", "P4", "P5"],
    "Gender": ["M", "F", "F", "M", "F"],
    "AppointmentType": ["Routine", "Emergency", "Routine",
                        "Follow-up", "Emergency"],
    "City": ["New York", "Los Angeles", "Chicago", "New York",
```

```

"Chicago"]
}

df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
print("\nDataset Info:")
df.info()

```

Output:

Original Dataset:

	Patient_ID	Gender	AppointmentType	City
0	P1	M	Routine	New York
1	P2	F	Emergency	Los Angeles
2	P3	F	Routine	Chicago
3	P4	M	Follow-up	New York
4	P5	F	Emergency	Chicago

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 5 entries, 0 to 4

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Patient_ID	5 non-null	object
1	Gender	5 non-null	object
2	AppointmentType	5 non-null	object
3	City	5 non-null	object

dtypes: object(4)

memory usage: 292.0+ bytes

- All columns are of type object (categorical/text).
- No missing values.
- The dataset has 5 rows and 4 columns.

2. Label Encoding

Label Encoding assigns a unique integer to each category in a column. We use `sklearn.preprocessing.LabelEncoder` to transform the three categorical columns.

```
from sklearn.preprocessing import LabelEncoder
```

```
# Create a copy of the original dataframe for label encoding
```

```
df_label = df.copy()
```

```
# Initialize label encoders for each column

le_gender = LabelEncoder()

le_appt = LabelEncoder()

le_city = LabelEncoder()


# Apply label encoding

df_label['Gender'] = le_gender.fit_transform(df_label['Gender'])

df_label['AppointmentType'] =
le_appt.fit_transform(df_label['AppointmentType'])

df_label['City'] = le_city.fit_transform(df_label['City'])


print("Dataset after Label Encoding:")

print(df_label)

print("\nMapping of categories to integers:")

print("Gender: M ->", le_gender.transform(['M'])[0], ", F ->",
le_gender.transform(['F'])[0])

print("AppointmentType: Routine ->", le_appt.transform(['Routine'])[0],
      ", Emergency ->", le_appt.transform(['Emergency'])[0],
      ", Follow-up ->", le_appt.transform(['Follow-up'])[0])
```

```
print("City: New York ->", le_city.transform(['New York'])[0],  
      ", Los Angeles ->", le_city.transform(['Los Angeles'])[0],  
      ", Chicago ->", le_city.transform(['Chicago'])[0])
```

Output:

Dataset after Label Encoding:

	Patient_ID	Gender	AppointmentType	City
0	P1	1	2	2
1	P2	0	0	1
2	P3	0	2	0
3	P4	1	1	2
4	P5	0	0	0

Mapping of categories to integers:

Gender: M -> 1 , F -> 0

AppointmentType: Routine -> 2 , Emergency -> 0 , Follow-up -> 1

City: New York -> 2 , Los Angeles -> 1 , Chicago -> 0

- Observation: Each category is replaced by an integer. The mapping is arbitrary (alphabetical order by default).
- The shape remains (5, 4) – no new columns are added.

3. One-Hot Encoding

One-Hot Encoding creates binary (0/1) columns for each category. We use `pandas.get_dummies()` to convert the categorical columns. The original columns are dropped, and the new dummy columns are concatenated with the non-categorical identifier (Patient_ID).

```
# Perform one-hot encoding on the categorical columns
```

```
df_onehot = pd.get_dummies(df, columns=['Gender',  
'AppointmentType', 'City'])
```

```
print("Dataset after One-Hot Encoding:")
```

```
print(df_onehot)
```

```
print("\nShape of one-hot encoded dataset:", df_onehot.shape)
```

Output:

Dataset after One-Hot Encoding:

	Patient_ID	Gender_F	Gender_M	AppointmentType_Emergency
0	P1	False	True	
	False			
1	P2	True	False	

True

2 P3 True False

False

3 P4 False True

False

4 P5 True False

True

AppointmentType_Follow-up AppointmentType_Routine
City_Chicago \

0 False True

False

1 False False

False

2 False True

True

3 True False

False

4 False False

True

City_Los Angeles City_New York

0 False True

1	True	False
2	False	False
3	False	True
4	False	False

Shape of one-hot encoded dataset: (5, 9)

Observation:

- Gender (2 categories) → 2 binary columns.
- AppointmentType (3 categories) → 3 binary columns.
- City (3 categories) → 3 binary columns.
- Total columns = 1 (Patient_ID) + 2 + 3 + 3 = 9.
- The values are Boolean (True/False), but can be converted to integers (0/1) if needed. In pandas, they behave as 1/0 in numeric operations.

We could also use `pd.get_dummies(..., drop_first=True)` to avoid multicollinearity, but the task asks to observe the expansion, so we keep all.

4. Comparative Analysis

Original Dataset:

Contains 5 rows and 4 columns. All columns are categorical (object type).

After Label Encoding:

The dataset still has 5 rows and 4 columns. The categorical values in the columns Gender, AppointmentType, and City have been replaced with integer codes (e.g., 0, 1, 2). No new columns are added; the structure remains the same, only the data type changes to numeric.

After One-Hot Encoding:

The dataset expands to 5 rows and 9 columns. Each original categorical column is replaced by multiple binary columns (one per unique category). Specifically:

Gender (2 categories) → 2 binary columns (Gender_F, Gender_M).

AppointmentType (3 categories) → 3 binary columns (AppointmentType_Emergency, AppointmentType_Follow-up, AppointmentType_Routine).

City (3 categories) → 3 binary columns (City_Chicago, City_Los Angeles, City_New York).

The original categorical columns are dropped, and only the Patient_ID remains as a non-encoded identifier.

- Label Encoding preserves the original number of columns but introduces arbitrary ordinal relationships.
- One-Hot Encoding significantly increases the feature space,

especially when a column has many unique categories.

When Each Method Is Appropriate

Label Encoding is suitable for:

- Ordinal categorical data where categories have a natural order (e.g., education level: "High School" < "Bachelor's" < "Master's").
- Tree-based models (decision trees, random forests) that can handle arbitrary integer labels without assuming order.
- Situations where memory or dimensionality is a concern (high-cardinality features).

One-Hot Encoding is suitable for:

- Nominal categorical data with no intrinsic order (e.g., Gender, City).
- Linear models, neural networks, and distance-based algorithms (e.g., k-NN, SVM) where arbitrary numeric labels could mislead the model by implying false order.
- When the number of categories is small, so the dimensionality increase is manageable.

Drawbacks of Label Encoding

- Implies ordinality: Assigning integers (e.g., 0,1,2) to nominal categories like cities suggests that "Chicago" (0) is somehow

less than "Los Angeles" (1) and "New York" (2), which is meaningless and can degrade model performance.

- Not suitable for linear models: The model may interpret the numeric values as having a linear relationship, leading to incorrect assumptions.
- Mapping is arbitrary: The integer assignment depends on the encoder's internal sorting (usually alphabetical), which can change if new categories appear.

Drawbacks of One-Hot Encoding

- Dimensionality explosion: If a categorical feature has many unique values (e.g., ZIP codes), the number of columns can become very large, causing memory issues and the "curse of dimensionality."
- Sparsity: Most values in the dummy columns are zero, which can be inefficient for some algorithms.
- Multicollinearity: In linear models, having all dummy variables for a feature can introduce perfect multicollinearity (the dummy variable trap). This can be avoided by dropping one category (`drop_first=True`).

Conclusion

Both encoding techniques transform categorical data into numerical form, but they do so in fundamentally different ways. Label

Encoding is simple and compact but risks introducing false ordinal relationships. One-Hot Encoding preserves the nominal nature of categories at the cost of increased dimensionality. The choice between them should be guided by the nature of the data (ordinal vs. nominal), the requirements of the machine learning algorithm, and the practical constraints of the dataset size.

It is common to use One-Hot Encoding for nominal features and Label Encoding (or similar ordinal encoding) for ordinal features. For high-cardinality features, alternatives like frequency encoding or embedding layers may be considered.